

Highly Scalable and Reliable Poll Widget System Design

Generated on 2025-08-10 02:41 UTC

Highly Scalable and Reliable Poll Widget System Design

Generated on 2025-08-10 02:41 UTC

One Pager Summary

Goal: Deliver an embeddable poll widget for any website that supports massive read and write scale with strong reliability, low latency, and privacy by design.

Users: Voters on publisher sites, site owners embedding the widget, administrators managing polls.

Core Value: Drop in JavaScript widget, instant results after voting, resilient and cost efficient backend that handles viral spikes.

North Star Metrics: Time to First Vote under 200 milliseconds, p95 write latency under 150 milliseconds per region, p95 read latency under 100 milliseconds via edge cache, system availability 99.95 percent or better, data durability eleven nines for aggregates and raw votes.

Scope: Create, configure, and distribute polls; record and revoke votes; fetch results; anti abuse controls; analytics for site owners.

Functional Requirements

Create polls with multiple options, start and end times, visibility, and tenant level settings.

Embed widget on any page via a single script tag with async load and auto initialization.

Record a vote for a poll option after the widget verifies eligibility and rate limits.

Remove a vote if the poll allows unvote or if owners revoke fraudulent votes.

Fetch results for a poll, returning per option tallies. Results are revealed after the user votes or when poll visibility allows.

Multi tenancy isolation per site owner with usage quotas, API keys, RBAC, and limits.

Optional identity binding: anonymous with device fingerprint and cookie, first party user id, or OIDC id via signed JWT.

Observability for tenants and platform: request volumes, error rates, conversion, and latency.

Administrative tooling for fraud review, manual adjustments with audit trail, and support workflows.

Non

Availability target 99.95 percent platform wide, 99.99 percent for edge read APIs.

Low latency: p95 read under 100 milliseconds from edge, p95 write under 150 milliseconds within a region.

Consistency model: Strongly consistent within a region for single poll tallies, eventually consistent cross region within 2 seconds.

Durability: eleven nines for vote events in object storage and six nines for counters via multi AZ databases.

Security and privacy: soc 2 type 2 preparedness, gdpr ccpa compliant data handling, data minimization and deletion tooling.

Scalability: Handle 200 thousand write requests per second worldwide with flash spikes up to 1 million write requests per second for 60 seconds, and 10 times read request volume at the edge.

Cost efficiency: Majority of reads served from edge caches or pre computed aggregates, object storage for raw events.

Architecture Overview

Client: Lightweight JavaScript snippet that injects a web component. The component fetches poll metadata from a low ttl edge cache, submits votes to the closest regional write api via anycast or geo dns, and opens a streaming channel for live result updates where supported.

Edge Layer: CDN with serverless edge functions for read APIs and token verification, short ttl caching for poll metadata and computed tallies, bot detection challenges, and request signing. Websocket support for fan out via edge pub sub.

Write Path Region

Api Gateway with WAF and DDoS shielding.

Stateless Vote Service in containers or serverless functions that authenticate, validate, and enqueue vote

commands.

Durable Messaging via a partitioned log service like Kafka or Kinesis to absorb spikes and order events by poll id.

Command Processor that materializes votes to storage and increments counters, idempotent by vote id and user context.

Anti Abuse Service that evaluates risk using device signals, ip reputation, velocity checks, and tenant rules.

Materialized Views service that maintains per poll counters in a fast key value store and publishes changes to a notification topic.

Results API that reads from the materialized store and edge cache. Writes invalidate or update cache using pub sub.

Storage

Raw Events: Append only vote and unvote events in object storage partitioned by date and tenant for compliance and analytics.

Operational Store: Document or relational store for polls, tenants, api keys, and settings.

Counters: Strongly consistent per key counters using a NoSQL store with conditional updates or relational database with row level counters.

Analytics: Data lake plus query engine to compute historical insights and anti abuse models.

Multi Region and Data Flow

Active active regions for write with sticky routing by user proximity. Cross region replication for operational data and periodic exchange of counter deltas. Global edge for reads.

Vote: Client signs and posts vote to regional API, service authenticates and deduplicates, enqueues command. Processor writes raw event and updates counters transactionally. Counter updates trigger invalidation and push updates to edge.

Fetch Results: Edge function returns cached tallies; on miss, fetches from regional Results API then caches with short ttl. If user has not voted, widget hides tallies until after vote per policy.

Technology Stack

Client: TypeScript, Web Components, ESM build with integrity attributes.

Edge: CDN with edge compute and key value cache, Websocket edge pub sub.

Gateway and Compute: Managed API Gateway or Envoy, containerized services on Kubernetes or serverless functions.

Messaging: Kafka or Kinesis with per poll id partitioning key.

Operational Store: Postgres with read replicas or DynamoDB equivalent, chosen per team expertise. Use optimistic concurrency controls for counters.

Materialized Counters: Redis with Redis Cluster plus periodic reconciliation from the log and durable store. Alternative is DynamoDB counters with conditional updates.

Object Storage: Cloud object storage with versioning and lifecycle policies.

Analytics: Data lake on object storage with Spark or serverless query, plus feature store for abuse detection.

Secrets and Identity: KMS, short lived tokens, OIDC for tenants, token binding to origin.

Observability: OpenTelemetry, Prometheus, Grafana, CloudWatch equivalent, centralized logging and trace sampling.

CI CD: Git based workflow, IaC with Terraform or CloudFormation, progressive delivery with canary and feature flags.

Data Model Sketch

Tenant: id, name, plan, quotas, status, createdAt.

Poll: id, tenantId, question, options array of id and text and order, startAt, endAt, visibility, allowUnvote, rateLimitPolicy, createdAt, version.

VoteEvent: id, pollId, optionId, tenantId, userId optional, deviceId optional, ipHash, ts, type VOTE or UNVOTE, riskScore, meta.

Tally: pollId, optionId, count, updatedAt, regionId, version.

AbuseCase: id, pollId, signals, action, reviewerId, createdAt.

API Design

Authentication for tenant admin APIs via OAuth client credentials or OIDC. Public widget uses signed short

lived tokens minted per poll and per origin to prevent abuse.

Idempotency via Idempotency Key header and server side de duplication by vote id and user context.

Version all endpoints under v1.

Public Widget APIs

POST v1 polls pollId votes: Request idempotencyKey, optionId, userToken, deviceFingerprint, consent flags.

Response accepted with voteReceiptId.

DELETE v1 polls pollId votes voteReceiptId: Request userToken. Response success.

GET v1 polls pollId results: Query afterVote only. Response per option tallies and lastUpdated.

Tenant Admin APIs

POST v1 polls, PATCH v1 polls pollId, GET v1 polls pollId, GET v1 polls pollId analytics, POST v1 polls pollId moderation actions.

Latency Budgets and Throughput

Edge read path goal p95 under 100 milliseconds for cached results. On cache miss, regional fetch adds 50 to 80 milliseconds intra region plus CDN backhaul.

Write path p95 under 150 milliseconds in region inclusive of authentication 20 milliseconds, enqueue 10 milliseconds, counter update 40 milliseconds, cache invalidation 10 milliseconds, response 20 milliseconds.

Throughput targets: steady 200 thousand writes per second globally with auto scaling on queue depth and consumer lag. Reads are ten times writes from edge caches.

SLA and SLOs

Platform availability 99.95 percent monthly, edge results 99.99 percent. Error budget policy of 0.5 percent.

RPO zero for raw events due to append to object storage before ack. RTO per region under 15 minutes with automatic failover.

Algorithms and Workflows

Idempotency: Compute a deterministic vote key using tenant id, poll id, user identity when known or device fingerprint plus a salt. Store a short ttl record to deduplicate retries.

Counter Consistency: Use a two phase approach. First update fast counter in Redis with a compare and set or Lua script ensuring per option atomicity. Second, persist the event and a durable counter snapshot periodically. Reconcile from the log on restart.

Reveal After Vote: Widget keeps a local flag upon successful vote. Edge function verifies a signed token claim voted equals true for five minutes window to reveal results without leaking to non voters.

Anti Abuse: Risk scoring uses velocity per ip prefix, device density per poll, ASN reputation, and tenant policies. High risk votes are accepted as shadow and excluded from counters pending review.

Unvote: Only allowed within window or per tenant policy. Processor will write an UNVOTE event and decrement counters idempotently.

Security Privacy and Compliance

PII minimization: default anonymous with hashed ip and coarse geolocation. Optional user id is pseudonymous. Clear consent text and lawful basis tracking.

Data retention: Raw events retained twelve months by default. Per tenant retention and deletion API. Right to be forgotten via user id or device id re keying.

Audit logs: Immutable logs for admin actions and moderation.

Encryption: TLS in transit, server side encryption with KMS at rest. Key rotation every ninety days.

Vendor risk: annual soc 2 type 2 audits, pen tests, secret scanning, SCA.

GDPR and CCPA: DPA with tenants, SCCs as needed, data processing records, regional hosting in EU on request, subject access and deletion workflows.

Multi Tenancy

Logical isolation by tenant id and row level security in the operational store. Rate limits and quotas per tenant.

Dedicated domains and API keys. Option for dedicated regional data residency and VPC isolation for enterprise plans.

Scaling and Sharding

Partition vote streams by poll id. Hot polls get dedicated partitions and consumers. Tally keys use poll id and option id for atomic updates. Redis cluster slots distribute load. Background rebalancing moves hot keys to isolated shards. Use write through cache and pub sub to edge to reduce origin reads.

CI CD and IaC

Source control trunk based development with short lived branches. Automated tests and security scans on every PR. Infrastructure defined in Terraform with separate workspaces per environment. Progressive delivery with canary and traffic shifting, automated rollback on SLO regression using metric based alerts. Feature flags to dark launch new anti abuse models.

Observability and Operations

Telemetry: traces for API and processing pipeline with propagation from client. Metrics: QPS, p50 p95 p99 latency, cache hit rate, consumer lag, error rates, drop rate due to abuse.

Alerting: multi channel alerts on SLO burn, queue lag, cache miss spikes, and regional error anomalies.

Runbooks and dashboards linked from alerts.

Chaos testing: fault injection at queue, store, and cache. Regional failover drills quarterly. Game days with realistic spikes.

Cost Monitoring: per tenant cost attribution, budgets, and anomaly detection.

Capacity Planning and Cost Estimates

Example scale for one region: steady 20 thousand writes per second per region with bursts to 100 thousand. Kafka three brokers class i3en storage, Redis cluster with nine shards, Postgres with three writer ready nodes and read replicas. Edge cache hit over 95 percent keeps origin read traffic under 5 thousand RPS. Estimated monthly cloud spend order of tens of thousands of dollars depending on provider and data egress; optimize via higher cache ttl for cold polls and object storage lifecycles.

Testing Strategy and Failure Drills

Unit tests for validators, idempotency, and counter logic. Property based tests for reconciliation. Contract tests for public and tenant APIs. Load tests with spike and soak scenarios. End to end tests with ephemeral environments per PR. Security testing with SAST, DAST, dependency scanning, and secrets scanning.

Failure drills simulate queue outage, Redis failover, database failover, and cache cold start. Validate zero data loss, bounded latency degradation, and automated recovery. Practice runbooks and MTTR capture.

Phased Build Plan

MVP two to four weeks: Single region write, edge cached reads, create poll, vote, results after vote, simple device based rate limit, basic dashboards, IaC, CI CD, and on call.

Phase 2 two months: Anti abuse service, idempotency receipts, unvote flow, analytics export, canary deploys, chaos tests, and multi AZ.

Phase 3 three to six months: Multi region active active, websocket live updates, enterprise tenant isolation and data residency, advanced analytics and anomaly detection, marketplace integrations.

Key Trade Offs and Risks

Edge versus origin complexity increases cache invalidation complexity; mitigation is strict cache keys and pub sub invalidation.

Counter accuracy versus cost; mitigation is per region strong consistency with eventual global convergence and expose last updated.

Hot poll hotspots; mitigation is auto splitting and dedicated partitions and predictive scaling from queue depth.

Abuse arms race; mitigation is model iteration, feedback loops, and layered defenses.